

Agent 当前真实架构说明

本文用于说明 /analysis 中 Agent 的当前真实形态，面向评审、汇报和产品沟通场景。

这不是一份未来方案，也不是工程内部的接口手册。它回答的是三个问题：

- 现在的 Agent 是什么
- 它当前怎么工作
- 为什么它比早期方案更收束、更可信

1. 文档定位

当前 Agent 已经不是一个“加在地图右侧的聊天框”，而是一套围绕城市空间分析任务组织起来的受控运行系统。

它的目标不是做开放式通用助手，而是把用户提出的问题，和当前地图范围、分析结果、工具能力、证据边界组合起来，产出一段可解释的自然回答。

在当前版本里，Agent 的默认任务可以概括为一句话：

基于当前地图分析上下文，继续回答用户问题，并在需要时调用少量工具补充证据。

这套设计的重点不在“让模型自由发挥”，而在：

- 问题入口清晰
- 工具调用受控
- 回答边界明确
- 过程可追踪
- 会话可恢复

2. 一页式核心判断

当前 Agent 已经完成了从“多角色串行交卷”到“单一路径自然回答”的收敛。

当前的核心特征有四点：

1. 主链路只有一条

用户问题统一进入 `turn / turn/stream` 主入口，不再维护独立实验链路。

2. 中段是受控工具循环

系统会先判断问题是否清晰、是否需要澄清，再按需进入工具循环，证据够了就停，不再默认做长链条规划。

3. 最终输出是自然回答

`answered` 状态的主契约只有 `answer`，系统不再把结构化栏目当作用户最终回复。

4. 后台仍保留完整可解释性

虽然用户看到的是自然文本，但后台仍保存工具轨迹、过程状态、证据引用、风险确认和会话上下文。

对评审来说，这意味着当前 Agent 的产品形态已经比较明确：

- 前台是“会继续回答问题的分析助手”
- 后台是“有工具治理和证据边界的分析运行时”

3. 当前 Agent 主链路

当前主链路可以概括为：

```
用户问题
-> 入口校验
-> 轻量门卫判断
-> 工具循环
-> 证据检查
-> 自然回答
-> 会话持久化
```

其中最关键的三个实现锚点是：

- `modules/agent/runtime.py`
- `modules/agent/providers/langgraph_react.py`
- `modules/agent/session_service.py`

3.1 入口阶段

主入口只有两个：

- `/api/v1/analysis/agent/turn`
- `/api/v1/analysis/agent/turn/stream`

输入的核心是：

- 用户消息
- 当前 `analysis_snapshot`
- `thinking_mode: quick | deep`
- 风险确认状态
- 上传附件引用

这里的 `thinking_mode` 只决定内部执行深度，不决定回答模板。

- `quick` 更偏复用已有证据、少量补工具
- `deep` 更偏多做一轮校验、把边界说得更稳

两者最终都回到同一个目标：继续回答用户问题。

3.2 门卫判断

进入工具阶段前，系统会先做一次轻量判断，确认：

- 问题是否足够清晰
- 当前范围和上下文是否能支撑执行
- 是否需要先向用户补充关键信息

如果问题不清晰，系统不会硬编造结论，而是进入澄清分支。

这一步的价值是把“是否能开工”提前判断掉，避免模型在不完整输入上自由发挥。

3.3 工具循环

通过门卫后，系统进入 LangGraph 风格的工具循环。

这一段不是一次性列出完整大计划，而是按轮次判断：

- 现有证据够不够
- 如果不够，最该先调什么工具
- 工具结果回来后是否还要继续
- 什么时候停止并进入回答

它的运行原则是：

- 优先复用已有快照和已有结果
- 只有确实缺证据时才调用新工具
- 证据足够时立即停止，不把分析拖成长流程

这使得 Agent 更像一个受控分析运行时，而不是一个无边界的自由推理器。

3.4 证据检查与回答生成

工具循环结束后，系统会做一次规则层面的证据检查，识别：

- 当前有哪些可用证据
- 哪些维度仍然缺失
- 哪些结论只能做方向性表达
- 哪些解释必须保留边界

随后，系统把这些证据整理成轻量回答底座，再生成最终自然回答。

当前回答生成的组织原则是：

- 先直接回答用户问题
- 再展开最关键的判断依据
- 在必要时补充边界或下一步

总结类问题会更完整一些，解释类和建议类问题会更聚焦一些，但都不再强制按固定栏目输出。

4. 当前前后端交互与回答契约

4.1 当前对外主契约

当前 Agent 对外主契约非常收束：

- 主请求：`/api/v1/analysis/agent/turn`
- 流式请求：`/api/v1/analysis/agent/turn/stream`
- answered 主输出：`output.answer`
- 执行深度开关：`thinking_mode: quick | deep`

非 answered 状态只保留两类产品语义：

- 澄清问题
- 风险确认

这意味着系统不再把结构化栏目当作用户回复契约，也不要求前端去拼装判断、证据、建议卡片。

4.2 后端统一生成 canonical assistant message

当前 assistant 的主回答内容由后端统一生成并持久化。

这带来两个直接收益：

1. 用户看到的 assistant 消息有唯一来源
刷新页面、切换历史、重新打开会话后，看到的回答不会因为前端重组逻辑不同而变化。
2. 过程和结果可以一起恢复
每条 assistant 消息不仅有最终文本，也带有对应的过程信息、工具轨迹和执行摘要。

换句话说，前端看到的是“回答的呈现”，不是“回答的再生产”。

4.3 前端当前负责什么

前端当前负责：

- 渲染消息内容
- 渲染思考过程和工具轨迹
- 管理会话切换与恢复
- 管理澄清、风险确认和继续分析交互

前端不再负责：

- 拼装结构化结论
- 复刻后端回答逻辑
- 决定工具治理规则

这让前后端职责分界比早期版本更清楚。

5. 从旧链路收敛到当前单一路径

当前架构不是一步到位得到的，而是从更重、更散的方案收束而来。可以用四组变化来理解。

5.1 双轨链路 -> 单一路径

早期系统同时维护主链路和独立实验链路，前后端都要理解两套入口、两套事件流和两套心智模型。

当前已经收束为统一主路径：

- 所有普通继续分析都走同一条 **turn** 主链路
- 中段工具循环作为主链路内部机制存在
- 不再保留并行的独立产品面

这让评审更容易理解“系统现在到底有几条路”，也降低了维护和解释成本。

5.2 串行交卷 -> 轻治理加工具循环

早期链路更像一套长流程审查系统：前段设定框架，中段按框架执行，后段再审一轮是否够答。

当前收敛后的逻辑更直接：

- 先判断能不能回答
- 再按需补证据
- 证据够了就回答

这让简单问题不再被拉进复杂的内部流程，也让系统更符合用户对“继续分析”的预期。

5.3 结构化 answered 契约 -> 单一自然文本契约

早期 answered 输出更像一份结构化交付物，系统内部和前端都围绕多块字段组织。

当前 answered 输出只保留自然文本主回答：

- 用户看到的是一段完整回答
- 系统内部继续保留过程和证据
- 结构化中间层不再塑造用户最终回复

这一步的意义是把“内部治理结构”和“用户可见表达”真正拆开。

5.4 continue-analysis 夹带报告语义 -> 继续回答问题

早期“继续分析”容易和报告写回、模块生成、结构化审查语义混在一起。

当前已经明确：

- continue-analysis 的默认目标就是继续回答当前问题
- deep 只是更强的证据检查，不是报告模式
- 报告、总结、选址、迭代解释等能力保留为独立旁支，不再反向塑造主回答链路

这一步让产品语义更纯，也更符合用户直觉。

6. 当前系统边界与不做什么

为了让 Agent 的能力可解释，当前系统刻意保留了一些边界。

6.1 它不是通用开放式 Agent

当前 Agent 只服务 /analysis 工作台里的空间分析语境。

它擅长的是：

- 基于当前地图范围继续分析
- 利用已有结果和工具补证据
- 对区域特征、空间结构、供需关系、选址机会做方向性判断

它不以“无限自由任务处理”为目标。

6.2 它不会把 GIS 指标直接外推成经营结果

当前系统明确保留一条硬边界：

- GIS 密度、分布、人口、夜光、路网等指标，可以支持空间画像和趋势判断
- 但不能被直接翻译成客流、消费力、营业额或经营收益

这不是能力缺失，而是产品可信性的必要边界。

6.3 deep 不是报告模式

当前 **deep** 的含义只是：

- 工具轮次可以更多
- 证据校验更严格
- 边界表达更完整

它不再意味着固定栏目输出，也不意味着自动生成报告模块。

6.4 主对话链路不等于全部 Agent 能力

当前系统里仍有总结、选址、专题解释、历史分析等旁支能力。

但这些能力不等于主对话链路本身。

当前文档关注的是普通 Agent turn 主链路，而不是把所有分析服务混成一个总系统。

7. 评审视角下的价值与可信性

从评审视角看，当前这套 Agent 的价值不在“更像一个万能助手”，而在“更像一个可以落在地图分析场景里的可信系统”。

它当前的可信性来自三个层面。

7.1 它更短

高频问题不再默认经过冗长的多角色流程，而是更直接地收敛到回答。

这意味着：

- 响应路径更短
- 解释更直接
- 产品体验更接近日常 AI 问答

7.2 它更稳

系统现在把高频交互语义收束得更明确：

- 继续分析就是继续回答问题
- 工具循环只在需要时补证据
- answered 只产出自然回答

这让前后端都围绕同一套契约工作，减少了用户看不见但系统内部很重的复杂度。

7.3 它仍然可解释

虽然用户主视图看到的是自然回答，但后台仍然保留：

- 思考阶段
- 工具轨迹
- 风险确认
- 证据引用
- 会话恢复信息

这使它不是“纯黑箱聊天”，而是一套可以被复盘的分析系统。

对地图分析工作台来说，这种“前台自然、后台可追溯”的平衡，比一味追求更强的自由推理更有产品价值。

8. 后续演进方向

当前架构已经基本收束，但仍有几条清晰的后续方向。

第一，继续提高回答质量。

当前自然回答已经替代结构化模板，但后续仍可继续优化不同类型问题的展开度、证据表达和边界语气。

第二，继续强化高频任务的稳定性。

总结类、解释类、建议类问题已经有明显不同的回答风格，后续仍可继续固化高频场景的证据组织方式。

第三，继续产品化过程展示。

当前过程信息已经保留得较完整，后续可以进一步把技术轨迹翻译成更容易理解的进度表达。

第四，继续明确主链路与旁支能力的分层。

主对话链路、专题服务、总结能力和报告类能力，后续可以在文档和产品呈现上继续拉开层次，减少误解。

9. 简短结论

当前 Agent 的真实状态可以概括为：

一个面向地图分析工作台的
单一路径、工具受控、证据有边界、
结果可恢复、前台自然回答的分析系统

它现在最重要的变化不是“能力越来越多”，而是“主链路终于变得清楚”：

- 用户问题统一进入主入口
- 中段按需补证据
- 最终只输出自然回答
- 后台继续保留轨迹、证据和恢复能力

这套设计更适合城市空间分析场景，也更适合向评审说明：

当前 Agent 不是一套还在发散的实验系统，而是一条已经收束成型、可以持续打磨质量的产品主链路。